

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 2

Stack List dalam Python



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

Genap 2024/2025

A. Tujuan Praktikum

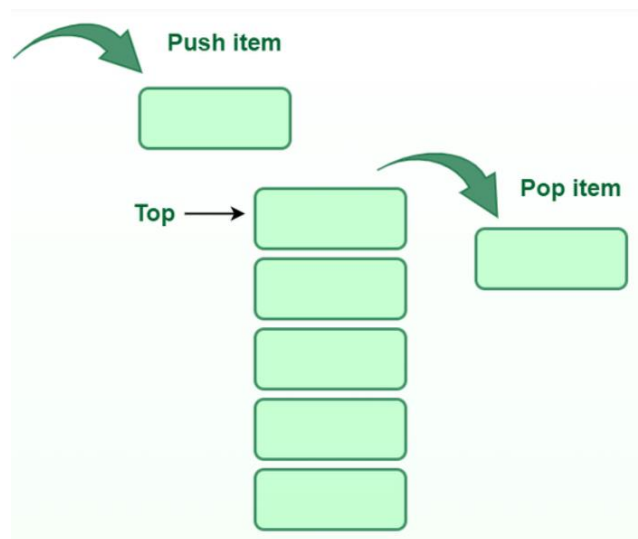
Agar setiap mahasiswa tahu cara mengaplikasikan struktur data stack dalam python dan mampu memahami algoritma struktur data stack

B. Dasar Teori

Stack adalah struktur data linier dimana penyisipan elemen baru dan penghapusan elemen yang terjadi pada ujung yang sama direpresentasikan sebagai bagian atas stack. Untuk mengimplementasikan stack, diperlukan penunjuk ke bagian atas stack, yaitu elemen terakhir yang disisipkan karena kita hanya dapat mengakses elemen yang berada di bagian atas tumpukan. LIFO startegi ini menyatkana bahwa elemen yang dimasukan terakhir akan keluar lebih dulu. Ini dapat mengambil tumpukan piring yang ditampangkan sebagai contoh kehidupan nyata.

Operasi Dasar pada Stack

- Push() untuk memasukan elemen ke dalam stack
- Pop() untuk menghapus elemen dari stack
- Top() menagmebalikan elemen teratas stack
- isEmpty() mengembalikan nilai benas jika tumpukan kosong jika tidak salah
- size() mengembalikan ukuran stack



Gambar 1.1

Jenis jenis Stack

- Fixed size stack : seperti Namanya, ukuran tetap memiliki ukuran yang tetap dan tidak dapat bertambah atau menyusut secara dinamis. Jika stack penuh dan upaya dilakukan untuk menambahkan elemen ke dalamnya.
- Dynamic size stack : ini dapat bertambah atau menyusut secara dinamis Ketika stack sudah penuh, secara otomatis ukurannya akan bertambah untuk menampung elemen baru, dan Ketika stack kosong.
- Infix to Postfix Stack : jenis stack ini digunakan untuk mengubah ekspresi infiks menjadi ekspresi postfix
- Stack Evaluasi Ekspresi : Jenis stack ini digunakan untuk mengevaluasi ekspresi postfix
- Stack Rekursi : Jenis stack ini digunakan untuk menyimpan nilai perhitungan program dan nilai registrasi dalam program computer, sehingga program dapat Kembali ke keadaan sebelumnya Ketika fungsi Kembali
- Balanced Parathesis Stack : Jenis stack ini digunakan untuk memeriksa keseimbangan tanda kurung dalam sebuah ekspresi
- Undo-Redo Stack : Jenis stack ini digunakan dalam program computer untuk memungkinkan pengguna membatalkan dan mengulangi Tindakan

Keuntungan Stack

- Implementasi yang mudah : Struktur data stack mudah diimplementasikan menggunakan array atau daftar tertaut, dan pengoperasiannya mudah dipahami dan diterapkan
- Pemanfaat memori yang efisien : Stack menggunakan blok memori yang berdekatan, membuatnya lebih efisien dalam pemanfaatan memori dibandingkan struktur data lainnya
- Waktu akses cepat : Struktur data stack menyediakan waktu akses cepat untuk menambahkan dan menghapus elemen saat elemen ditambahkan dan dihapus dari atas stack
- Membantu dalam pemanggilan fungsi : Struktur data stack digunakan untuk menyimpan pemanggilan fungsi rekursif secara efisien
- Digunakan dalam desain kompailer : Struktur data stack digunakan dalam desain kompailer untuk penguraian dan analisis sintaks Bahasa pemrograman

Kekurangan stack

- Kapasitas terbatas : struktur data stack memiliki kapasitas terbatas karena hanya dapat menampung sejumlah elemen tetap. Jika stack penuh, penambahan elemen baru dapat mengakibatkan stack meluap, yang menyebabkan hilangnya data
- Tidak ada akses acak : Struktur data stack tidak mengizinkan akses acak elemennya, dan hanya mengizinkan penambahan dan penghapusan elemen dari atas stack. Untuk mengakses elemen di tengah stack, semua elemen di atasnya harus di hapus.
- Manajemen memori : Struktur data stack menggunakan blok memori yang berdekatan, yang dapat mengakibatkan fragmentasi memori jika elemen sering ditambahkan dan dihapus
- Stack overflow dan underflow : Struktur data stack dapat mengakibatkan stack overflow jika terlalu banyak elemen yang dimasukkan ke dalam stack, dan dapat mengakibatkan stack underflow jika terlalu banyak elemen yang dikeluarkan dari stack
- Batasan pemanggilan fungsi rekursif : Meskipun struktur data stack mendukung pemanggilan fungsi rekursif, terlalu banyak pemanggilan fungsi rekursif dapat menyebabkan stack overflow, yang mengakibatkan penghentian program.

C. Hasil dan Pembahasan

1. Stack dengan List

```
#Stack dengan list
stack = []

stack.append('n')
stack.append('o')
stack.append('p')
stack.append('a')
stack.append('l')

print('initial stack')
print(stack)

print('\nElemenets popped from stack:')
print(stack.pop())
print(stack.pop())
print(stack.pop())

print('\nStack after element are popped:')
print(stack)
```

Gambar 1.2

- Pertama kita membuat stack kosong dengan menggunakan list Python, yang digunakan untuk menyimpan elemen stack nantinya.
- Selanjutnya menambahkan elemen ke dalam stack menggunakan metode append
- Mencetak judul initial stack dan isi dari stacknya
- Mencetak pesan popped untuk menandakan akan ada value elemen yang akan dihapus
- Selanjutnya menghapus dan mencetak elemen teratas dari stack menggunakan pop
- Mencetak popped untuk menandakan bahwa operasi telah dilakukan dan menampilkan isi stack setelah elemen dihapus.

```
initial stack
['n', 'o', 'p', 'a', 'l']

Elemenets popped from stack:
l
a
p

Stack after element are popped:
['n', 'o']
```

Gambar 1.3

- Gambar diatas hasil dari kode pada gambar 1.2

2. Stack dengan Collections.deque

```
#Stack dengan collections.deque
from collections import deque

stack = deque()

#Menambahkan data dengan append

stack.append('n')
stack.append('0')
stack.append('p')
stack.append('a')
stack.append('l')

print('initial stack')
print(stack)

print('\nElemenets popped from stack:')
print(stack.pop())
print(stack.pop())
print(stack.pop())

print('\nStack after element are popped:')
print(stack)
```

Gambar 1.4

- Pertama mengimport kelas deque dari module collections
- Membuat objek deque yang akan digunakan sebagai stack dengan awalan kosong
- Menambahkan elemen ke dalam stack menggunakan metode append
- Mencetak pesan untuk menandakan bahwa akan ada elemen yang akan dihapus
- Menghapus dan mencetak elemen tertatas dari stack menggunakan pop
- Mencetak pesan bahwa operasi popped telah dilakukan

```
initial stack
deque(['n', '0', 'p', 'a', 'l'])

Elemenets popped from stack:
l
a
p

Stack after element are popped:
deque(['n', '0'])
```

Gambar 1.5

- Gambar diatas adalah hasil dari kode gambar 1.4

3. Stack dengan queue.LifoQueue

```
#Stack dengan queue.LifoQueue

from queue import LifoQueue

stack = LifoQueue(maxsize=5)

print(stack.qsize())

stack.put('n')
stack.put('o')
stack.put('p')
stack.put('a')
stack.put('l')

print("full:",stack.full())
print("size",stack.qsize())

print('\nElements popped from stack:')
print(stack.get())
print(stack.get())
print(stack.get())

print("\nEmpty:",stack.empty())
```

Gambar 1.6

- Pertama mengimport kelas LifoQueue dari modul queue
- Membuat stack dengan objek LifoQueue yang memiliki maxsize 5
- Mencetak qsize tetapi disana masih belum ditambahkan elemen nya kedalam stack jadi masih kosong
- Menambahkan elemen stack dengan put
- Memastikan apakah stack sudah penuh apa belum, dan jika belum penuh akan muncul full = false tetapi tidak eror
- Selanjutnya menghapus dan mencetak elemen menggunakan metode get
- Baris terakhir mencetak apakah stack kosong setelah operasi pop. Jika stack masih ada elemennya maka output akan false

```
initial stack
deque(['n', 'o', 'p', 'a', 'l'])

Elements popped from stack:
l
a
p

Stack after element are popped:
deque(['n', 'o'])
```

Gambar 1.7

- Gambar diatas adalah hasil kode dari gambar 1.6

4. Stack dengan Single Linkedlist

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self):
        self.head = Node(None)
        self.size = 0

    def __str__(self):
        cur = self.head.next
        out = ""
        while cur:
            out += str(cur.value) + ">"
            cur = cur.next
        return out[1:]

    def getSize(self):
        return self.size

    def isEmpty(self):
        return self.size == 0

    def peek(self):
        if self.isEmpty():
            raise Exception("Peeking from an empty stack")
        return self.head.next.value

    def push(self, value):
        node = Node(value)
        node.next = self.head.next
        self.head.next = node
        self.size += 1

    def pop(self):
        if self.isEmpty():
            raise Exception("Popping from an empty stack")
        remove = self.head.next
        self.head.next = self.head.next.next
        self.size -= 1
        return remove.value

if __name__ == "__main__":
    stack = Stack()
    for i in range(1, 11):
        stack.push(i)
        print(f"Stack: {stack}")

    for i in range(1, 0):
        remove = stack.pop()
        print(f"Pop: {remove}")
        print(f"Stack: {stack}")
```

Gambar 1.8

- Pertama membuat kelas node
- Selanjutnya membuat class stack. Didalam class stack terdapat banyak fungsi yang pertama adalah init untuk menunjuk dirinya sendiri dengan value 0
- Fungsi str agar nilai node nya terhubung secara berurutan
- Fungsi getSize untuk mengembalikan ukuran stack
- Fungsi isEmpty ini akan menampilkan true jika stack masih kosong
- Fungsi peek untuk mengembalikan nilai node paling atas dalam stack tanpa menghapus nodenya. Jika stack kosong, akan menghasilkan false
- Fungsi push untuk menambahkan node baru dengan nilai yang diberikan ke bagian atas stack
- Fungsi pop untuk menghapus stack paling atas dan mengembalikan nilainya dan ukuran stack akan berkurang 1
- Pada baris if name akan dieksekusi jika file ini dijalankan secara langsung. Dalam block ini objek stack akan menambahkan beberapa node ke stack dengan push kemudian menghapus menggunakan metode pop dan mencetak node yang dihapus


```
Stack: 10->9->8->7->6->5->4->3->2->1  
Pop: 10  
Pop: 9  
Pop: 8  
Pop: 7  
Pop: 6  
Stack: 5->4->3->2->1
```

Gambar 1.9

- Gambar diatas adalah hasil dari kode gambar 1.8

D. Kesimpulan

Dari praktikum diatas kita menyimpulkan bahwa struktur data stack dapat mempermudah pekerjaan kita jika digunakan dengan benar dan tahu Batasan saat menggunakannya. Dan kita sebagai mahasiswa juga lebih tahu detailnya tentang struktur data stack dan diharapkan mampu mengimplementasikannya di kehidupan sehari hari atau bahkan dalam membuat sebuah projek akhir nantinya

E. Daftar Pustaka

<https://www.geeksforgeeks.org/applications-advantages-and-disadvantages-of-stack/?ref=lbp>